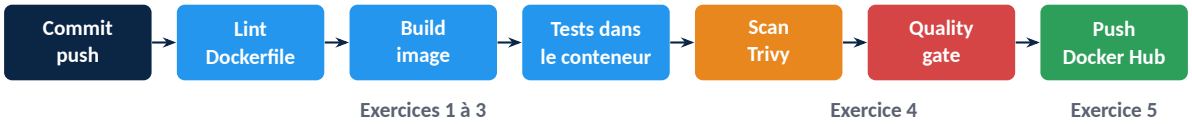
TP DEVOPS 4 · DOCKER

Dockeriser, versionner, scanner et publier une image

Contexte et objectifs

Vendredi, 17 h. Votre équipe vient de terminer une petite API. Elle fonctionne sur votre machine… mais pas sur celle du collègue, et personne ne sait quelle version tourne en production. Vous allez régler cela avec Docker : **empaqueter** l'application dans une image, **numéroter** chaque version, **vérifier** automatiquement sa qualité et sa sécurité, puis la **publier** sur Docker Hub, le tout sans intervention manuelle.

Ce TP est **indépendant** des TP précédents : tout est fourni, vous n'avez besoin d'aucun travail antérieur. Côté GitHub, vous ferez seulement des commits et des push simples sur `main` : l'essentiel se passe dans VS Code et dans Docker.



À la fin de la séance, vous saurez :

- écrire un `Dockerfile` propre (utilisateur non-root, healthcheck, `.dockerignore`) et construire une image ;
- versionner vos images avec des tags (`1.0.0`, `1.0`, `latest`) ;
- configurer un pipeline GitHub Actions qui construit et teste l'image ;
- scanner l'image avec **Trivy**, générer des rapports et **bloquer** le pipeline si une vulnérabilité grave est trouvée
- publier l'image sur Docker Hub sans jamais écrire un mot de passe dans le code.

0 Préparation

1. **Vérifier l'environnement** dans un terminal (celui de VS Code convient) :

```
docker --version
docker run hello-world
git --version
```

   Dans VS Code, installez l'extension **Docker** (Microsoft) si ce n'est pas fait.

2. **Docker Hub** : créez un compte, puis un *jeton d'accès* : Account settings → Personal access tokens → Generate new token (droits *Read & Write*). Copiez-le tout de suite : il ne s'affiche qu'une fois.
3. **GitHub** : créez un dépôt **public** nommé `ministatus-docker`, clonez-le, puis ouvrez le dossier dans VS Code (`code .`).
4. **Secrets** : dans le dépôt, Settings → Secrets and variables → Actions → New repository secret. Créez deux secrets : `DOCKERHUB_USERNAME` (votre identifiant Docker Hub) et `DOCKERHUB_TOKEN` (le jeton).
5. **Kit de départ** : copiez dans le dépôt les fichiers `app.py`, `test_app.py`, `requirements.txt` et `.gitignore`

(annexe A). Faites votre premier commit depuis VS Code (onglet *Source Control*, message `chore: application de depart`, puis *Sync Changes*), ou en ligne de commande :

```
git add .
git commit -m "chore: application de depart"
git push
```

Règle d'or : un secret ne vit que dans GitHub Secrets

Le jeton Docker Hub ne doit **jamais** apparaître dans un fichier du dépôt, dans un message de commit, dans le `Dockerfile` ni dans le YAML. Le workflow le lit uniquement par son nom (`secrets.DOCKERHUB_TOKEN`). C'est le même principe que le garde-fou « masquer les secrets » : une valeur sensible ne circule pas en clair.

Vous devez voir

Le message « Hello from Docker! » après `docker run hello-world`, et un dépôt GitHub contenant les 4 fichiers de départ.

1 Création de l'image

Rappel du cours

Une **image** est un modèle immuable construit à partir d'un `Dockerfile` ; un **conteneur** est une instance en cours d'exécution de cette image. Instructions vues en cours : `FROM` (image de base), `RUN` (commande pendant le build), `COPY` (copier des fichiers), `CMD` (commande de démarrage).

1. Créez à la racine un fichier `.dockerignore` (ce qui ne doit pas entrer dans l'image) :

`.dockerignore`

```
.git
.github
__pycache__/
*.pyc
.venv/
trivy-report.*
docker-compose.yml
```

2. Créez le fichier `Dockerfile` :

`Dockerfile`

```
FROM python:3.12-slim

ARG APP_VERSION=dev
ENV APP_VERSION=${APP_VERSION} \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN useradd --create-home appuser
USER appuser

EXPOSE 5000

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s \
  CMD python -c "import urllib.request as u; u.urlopen('http://localhost:5000/health')"

CMD ["gunicorn", "-b", "0.0.0.0:5000", "app:app"]
```

3. Construisez l'image, puis lancez un conteneur :

```
docker build -t ministatus:dev .
docker images ministatus
docker run -d --name ms -p 5000:5000 ministatus:dev
docker ps
```

4. Dans le navigateur, ouvrez `http://localhost:5000/health` puis `http://localhost:5000/hello/VotrePrenom`.
5. Inspectez le conteneur, puis arrêtez-le :

```
docker logs ms
docker exec ms whoami
docker stop ms
docker rm ms
```

6. Lancez les tests *dans* un conteneur (l'image contient l'appli et ses tests) :

```
docker run --rm ministatus:dev python -m unittest -v
```

7. Commit et push : `feat: Dockerfile et dockerignore`.

Pourquoi cet ordre dans le Dockerfile ?

`requirements.txt` est copié *avant* le reste du code : tant que les dépendances ne changent pas, Docker réutilise la couche en cache et le build est beaucoup plus rapide. `USER appuser` évite de faire tourner l'application en `root` dans le conteneur.

Vous devez voir

`{"status":"ok"}` dans le navigateur, `appuser` comme réponse à `whoami`, et 3 tests `OK` dans le conteneur. Après une trentaine de secondes, `docker ps` peut afficher `(healthy)` pendant que le conteneur tourne.

2 Versionnement des images

Versionnement sémantique (SemVer)

`MAJEUR.MINEUR.CORRECTIF` : `1.0.0` → `1.0.1` pour un correctif, `1.1.0` pour une nouvelle fonctionnalité, `2.0.0` pour un changement incompatible.

1. Construisez la version `1.0.0` avec trois tags en une seule commande :

```
docker build --build-arg APP_VERSION=1.0.0 -t ministatus:1.0.0 -t ministatus:1.0 -t ministatus:latest
      .
docker images ministatus
```

   Observez : les trois tags ont le **même** `IMAGE ID`.

2. Lancez la version `1.0.0` et vérifiez `http://localhost:5000/version`, puis arrêtez-la :

```
docker run -d --name ms -p 5000:5000 ministatus:1.0.0
docker stop ms
docker rm ms
```

3. Simulez un correctif : construisez la version `1.0.1` en redéplaçant `1.0` et `latest` :

```
docker build --build-arg APP_VERSION=1.0.1 -t ministatus:1.0.1 -t ministatus:1.0 -t ministatus:latest
      .
docker images ministatus
```

   Observez : `1.0.0` pointe toujours vers l'ancienne image, tandis que `1.0` et `latest` ont **avancé**.

Tag	Rôle	Change ?
`1.0.1`	Version exacte, reproductible	Jamais
`1.0`	Dernier correctif de la série 1.0	À chaque correctif
`latest`	Dernière version publiée	À chaque version

Attention

En production, on déploie un **tag exact** (`1.0.1`), jamais `latest` : sinon, impossible de savoir quelle version tourne ni de revenir en arrière. Le numéro de version est aussi injecté dans l'image (`APP_VERSION`) et visible sur `/version`.

3 Pipeline CI : lint, build, tests

Rappel GitHub Actions

Un workflow est un fichier YAML dans `.github/workflows/`. Il contient des **déclencheurs** (`on:`), des **jobs** et des **étapes**.

1. Créez le fichier `.github/workflows/docker.yml` :

`.github/workflows/docker.yml`

```yaml
name: Docker CI

on:
  push:
    branches: [main]
    tags: ["v*"]
  pull_request:
    branches: [main]

env:
  IMAGE: ministatus

jobs:
  build-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Lint du Dockerfile
        uses: hadolint/hadolint-action@v3.1.0
        with:
          dockerfile: Dockerfile
          failure-threshold: warning

      - name: Build de l'image
        run: docker build --build-arg APP_VERSION=ci -t ${{ env.IMAGE }}:ci .

      - name: Tests dans le conteneur
        run: docker run --rm ${{ env.IMAGE }}:ci python -m unittest -v
```

2. Commit `ci: pipeline docker (lint, build, tests)` et push.
3. Sur GitHub, ouvrez l'onglet **Actions** : le workflow démarre. Attendez le ✓ **vert**, ouvrez le job et dépliez chaque étape. **Prenez une capture.**

- **Hadolint** analyse le `Dockerfile` et signale les mauvaises pratiques avant même le build.
- Les tests tournent *dans* le conteneur : on valide l'image réelle, pas seulement le code.
- Le même pipeline s'exécutera à chaque push sur `main`, sur les tags `v*` et sur les Pull Requests.

Vous devez voir

Un job `build-scan` vert avec 4 étapes réussies : checkout, lint, build, tests.

4 Scan Trivy, rapports et quality gate

Pourquoi scanner une image ?

Une image embarque un système et des bibliothèques. **Trivy** compare leurs versions à une base de vulnérabilités connues (CVE) et indique pour chacune la gravité (`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`) et la version corrigée. Le **quality gate** est la règle qui fait *échouer* le pipeline au-dessus d'un seuil.

1. Ajoutez ces étapes **à la suite** de l'étape « Tests dans le conteneur » (même indentation) :

`.github/workflows/docker.yml (suite)`

```
      - name: Rapport Trivy (JSON)
        uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: ${{ env.IMAGE }}:ci
          format: json
          output: trivy-report.json
          exit-code: "0"

      - name: Rapport Trivy (HTML)
        uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: ${{ env.IMAGE }}:ci
          format: template
          template: "@/contrib/html.tpl"
          output: trivy-report.html
          exit-code: "0"

      - name: Resume dans la page du run
        if: always()
        run: |
          echo "### Rapport Trivy" >> "$GITHUB_STEP_SUMMARY"
          for s in CRITICAL HIGH MEDIUM; do
            n=$(jq "[.Results[]?.Vulnerabilities[]? | select(.Severity==\"$s\")] | length" trivy-
    report.json)
            echo "- $s : $n" >> "$GITHUB_STEP_SUMMARY"
          done

      - name: Publier les rapports
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: trivy-reports
          path: trivy-report.*

      - name: Quality gate (HIGH et CRITICAL)
        uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: ${{ env.IMAGE }}:ci
          format: table
          severity: HIGH,CRITICAL
          vuln-type: library
          ignore-unfixed: true
          exit-code: "1"
```

2. Commit `ci: scan Trivy, rapports et quality gate` et push.
3. **Le pipeline passe au vert.** Ouvrez le job : l'étape « Quality gate » ne signale aucune vulnérabilité `HIGH` ou `CRITICAL` dans les bibliothèques Python, car le fichier `requirements.txt` fourni utilise des versions à jour. **Capture du ✓ vert et de l'étape « Quality gate ».**
4. Retournez sur la page du run : le **Résumé** indique le nombre de vulnérabilités par gravité. En bas, section *Artifacts*, téléchargez `trivy-reports` et ouvrez `trivy-report.html` dans le navigateur. **Capture du rapport.**

Deux détails du workflow

1) Les rapports sont générés et publiés *avant* le gate (avec `if: always()`) : ainsi, même quand le gate échoue, vous pouvez lire les rapports. **2)** Le gate porte sur les *bibliothèques Python* (`vuln-type: library`), c'est-à-dire ce que votre équipe maîtrise. Les paquets du système de base restent visibles dans le rapport complet ; on les corrige en mettant à jour l'image de base.

Vous devez voir

Un run vert dont l'étape « Quality gate » est réussie, un Résumé qui indique le nombre de vulnérabilités par gravité, et un rapport HTML lisible dans les artefacts.

5 Publication sur Docker Hub (CD)

Docker Hub

Registre en ligne pour stocker et partager des images. Une image y est identifiée par `utilisateur/nom:tag`, par exemple `monuser/ministatus:1.0.0`. On publie **uniquement** des versions numérotées, jamais chaque commit.

1. Ajoutez ce deuxième job **à la fin** du fichier (au même niveau que `build-scan`, 2 espaces d'indentation) :

`.github/workflows/docker.yml (suite)`

```
  publish:
    needs: build-scan
    if: startsWith(github.ref, 'refs/tags/v')
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Connexion a Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Calcul des tags
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ secrets.DOCKERHUB_USERNAME }}/ministatus
          tags: |
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}
            type=raw,value=latest

      - name: Build et push
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          build-args: APP_VERSION=${{ steps.meta.outputs.version }}
```

2. Commit `ci: publication sur Docker Hub` et push. Dans Actions, constatez que le job `publish` est **ignoré** (*skipped*) : il n'y a pas de tag.

3. Publiez votre première version : créez et poussez un tag Git.

```
git tag v1.0.0
git push origin v1.0.0
```

4. Dans Actions, suivez le run du tag : `build-scan` (✓) puis `publish` (✓). Ouvrez les logs de l'étape de connexion : le nom d'utilisateur apparaît sous la forme `***`. **Capture.**
5. Sur `hub.docker.com`, ouvrez votre dépôt `ministatus` : les tags `1.0.0`, `1.0` et `latest` sont là. **Capture.**
6. **Vérifiez** que l'image publiée fonctionne : supprimez la copie locale, puis récupérez-la depuis Docker Hub (remplacez `VOTRE_USER`) :

```
docker rmi ministatus:1.0.0
docker pull VOTRE_USER/ministatus:1.0.0
docker run --rm -p 5000:5000 VOTRE_USER/ministatus:1.0.0
```

Ouvrez `http://localhost:5000/version` : la réponse doit être `1.0.0`. **Capture.**

Le gate protège la publication

`needs: build-scan` signifie que `publish` ne démarre que si le scan et les tests ont réussi : **aucune image avec une vulnérabilité grave ne peut atteindre Docker Hub**. Vérifiez aussi que le jeton n'est écrit nulle part : la commande `git grep -i dockerhub` ne doit afficher que des *noms* de secrets dans le YAML, jamais une valeur.

Vous devez voir

Trois tags sur Docker Hub, et la réponse `1.0.0` sur `/version` depuis l'image téléchargée.

6 Automatisation

1. **Docker Compose** : créez `docker-compose.yml` pour lancer l'application avec une seule commande :

`docker-compose.yml`

```
services:
  ministatus:
    build:
      context: .
      args:
        APP_VERSION: local
    image: ministatus:local
    ports:
      - "5000:5000"
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request as u; u.urlopen('http://localhost:5000/
    health')"]
      interval: 15s
      timeout: 3s
      retries: 3
```

```
docker compose up -d --build
docker compose ps
docker compose logs
docker compose down
```

2. **Scan planifié** : dans `docker.yml`, remplacez le bloc `on:` par celui-ci. Le pipeline (donc le scan) se relancera

chaque lundi, même sans nouveau commit, car de nouvelles CVE sont publiées tous les jours :

```yaml
on:
  push:
    branches: [main]
    tags: ["v*"]
  pull_request:
    branches: [main]
  schedule:
    - cron: "0 6 * * 1"
  workflow_dispatch:
```

3. **Mises à jour automatiques** : créez `.github/dependabot.yml`. Dependabot proposera des Pull Requests quand l'image de base ou une dépendance Python a une nouvelle version :

`.github/dependabot.yml`

```yaml
version: 2
updates:
  - package-ecosystem: "docker"
    directory: "/"
    schedule:
      interval: "weekly"
  - package-ecosystem: "pip"
    directory: "/"
    schedule:
      interval: "weekly"
```

4. Commit `ci: scan planifie et dependabot`, push. Dans Actions, lancez le workflow à la main avec *Run workflow*. **Capture de `docker compose ps`.**

Ce qui est automatisé	Comment
Lint, build, tests	à chaque push et Pull Request
Scan et rapports	à chaque exécution, et chaque lundi
Blocage des failles graves	quality gate (`exit-code: 1`)
Numérotation des tags	`docker/metadata-action` à partir du tag Git
Publication	job `publish`, uniquement sur un tag `v*`
Mise à jour des versions	Dependabot

Pour aller plus loin (bonus)

- **Nexus** comme registre privé (en local) : `docker run -d -p 8081:8081 -p 8082:8082 --name nexus sonatype/nexus3`. Le mot de passe initial de `admin` est dans `/nexus-data/admin.password` (`docker exec nexus cat /nexus-data/admin.password`). Créez un dépôt *docker (hosted)* avec le port HTTP 8082, puis `docker login localhost:8082`, `docker tag ministatus:1.0.0 localhost:8082/ministatus:1.0.0` et `docker push localhost:8082/ministatus:1.0.0`. (Les exécuteurs GitHub ne peuvent pas joindre un Nexus qui tourne sur votre machine.)
- **Build multi-étapes** : séparez la construction et l'exécution dans le `Dockerfile` et comparez la taille des images avec `docker images`.
- **SBOM** : générez l'inventaire des composants de l'image avec Trivy (format `cyclonedx`) et publiez-le comme artefact.
- **Preuve du gate** : remettez volontairement d'anciennes versions dans `requirements.txt` (`Flask==2.2.2` et `Werkzeug==2.2.2`), commit, poussez le tag `v1.0.2` : `publish` est ignoré, rien n'est publié. Supprimez ensuite le tag (`git push origin --delete v1.0.2` puis `git tag -d v1.0.2`) et restaurez les bonnes versions.

À rendre

> **Un PDF** contenant le lien de votre dépôt GitHub, vos captures numérotées et les réponses ci-dessous. **Chaque capture est accompagnée d'une phrase** qui explique ce qu'elle prouve et à quelle étape elle correspond.
>
> **Captures attendues :** (1) `docker images` avec les tags ; (2) l'application dans le navigateur ; (3) pipeline vert (Ex. 3) ; (4) pipeline vert avec l'étape « Quality gate » (Ex. 4) ; (5) rapport HTML Trivy ; (6) tags sur Docker Hub ; (7) `/version` depuis l'image téléchargée.

Questions

1. Quelle est la différence entre une image et un conteneur ?
2. Pourquoi le `Dockerfile` crée-t-il un utilisateur non-root ? Quel risque évite-t-on ?
3. Pourquoi ne pas déployer en production avec le seul tag `latest` ?
4. Que fait le quality gate et pourquoi les rapports sont-ils publiés *avant* lui ?
5. Pourquoi le jeton Docker Hub est-il stocké dans GitHub Secrets et non dans le dépôt ?

Annexe A : fichiers de départ

`app.py`

```python
import os

from flask import Flask, jsonify

app = Flask(__name__)


@app.get("/health")
def health():
    return jsonify(status="ok")


@app.get("/version")
def version():
    return jsonify(version=os.getenv("APP_VERSION", "dev"))


@app.get("/hello/<nom>")
def hello(nom):
    return jsonify(message=f"Bonjour {nom}")
```

`test_app.py`

```python
import unittest

from app import app


class TestApp(unittest.TestCase):
    def setUp(self):
        self.client = app.test_client()

    def test_health(self):
        rep = self.client.get("/health")
        self.assertEqual(rep.status_code, 200)
        self.assertEqual(rep.get_json(), {"status": "ok"})

    def test_version_presente(self):
        rep = self.client.get("/version")
        self.assertIn("version", rep.get_json())

    def test_hello(self):
        rep = self.client.get("/hello/Sara")
        self.assertEqual(rep.get_json()["message"], "Bonjour Sara")
```

`requirements.txt`

```
Flask==3.1.1
Werkzeug==3.1.3
gunicorn==23.0.0
```

`.gitignore`

```
__pycache__/
*.pyc
.venv/
trivy-report.*
```

> **Copier-coller depuis un PDF**
>
> Copier du code depuis un PDF peut perdre l'indentation (fatal en Python et en YAML). Utilisez de préférence les fichiers du kit de départ ; sinon, vérifiez l'indentation (4 espaces en Python, 2 espaces en YAML) avant de valider.

Annexe B : aide-mémoire Docker

Commande	Rôle
`docker build -t nom:tag .`	Construire une image
`docker images`	Lister les images
`docker run -d --name x -p 5000:5000 img`	Lancer un conteneur
`docker ps / docker ps -a`	Conteneurs actifs / tous
`docker logs x`	Voir les journaux
`docker exec x whoami`	Exécuter une commande dans le conteneur
`docker stop x / docker rm x`	Arrêter / supprimer
`docker rmi img`	Supprimer une image
`docker pull / docker push`	Télécharger / publier
`docker compose up -d --build`	Lancer les services Compose
`docker compose down`	Les arrêter